

TP 2 — Conditions et boucles en pratique

Semaine 2 ■ Fichiers dans NSI/TP02/ ■ 2 h

Objectifs

- Écrire des programmes complets combinant `if`, `while`, `for`
- Utiliser `input()` pour rendre les programmes interactifs
- Prendre le réflexe de **tester** son programme sur plusieurs cas

Nouveaux outils. `input("question")` renvoie **une chaîne** ; pour un nombre : `n = int(input("Nombre ? "))`. `import random` puis `random.randint(1, 100)` : entier aléatoire entre 1 et 100.

Partie A — Conditions

(30 min)

`mention.py` — Demander une note sur 20 et afficher : « Félicitations » (≥ 16), « Bien » (≥ 14), « Assez bien » (≥ 12), « Admis » (≥ 10), « Non admis » sinon. **Tests obligatoires** : 20, 16, 15.9, 10, 9.5, 0. Noter les résultats en commentaire.

`triangle.py` — Demander trois longueurs `a`, `b`, `c` et dire si le triangle est équilatéral, isocèle ou quelconque. ★ Bonus : vérifier d'abord qu'il est constructible (chaque côté inférieur à la somme des deux autres).

Partie B — Boucles bornées

(30 min)

`table.py` — Demander un entier `n` et afficher sa table de multiplication de $n \times 1$ à $n \times 10$, une ligne par produit, sous la forme `7 x 3 = 21`.

`somme.py` — Demander `n` et afficher la somme $1 + 2 + \dots + n$. Vérifier avec la formule $n(n+1)/2$ pour $n = 100$ (réponse : 5050).

`fizzbuzz.py` — Afficher les entiers de 1 à 50, en remplaçant les multiples de 3 par « Fizz », les multiples de 5 par « Buzz », les multiples des deux par « FizzBuzz ». **Réfléchir à l'ordre des tests !**

Partie C — Le juste prix

(40 min)

`justeprix.py` — L'ordinateur choisit un nombre au hasard entre 1 et 100. Le joueur propose des nombres ; le programme répond « Plus grand ! » ou « Plus petit ! » jusqu'à la victoire, puis affiche le nombre d'essais.

Étapes conseillées :

1. Version sans hasard (`secret = 42`) pour mettre au point la boucle.
2. Ajouter `random.randint`.
3. Compter les essais.
4. ★ Bonus : limiter à 7 essais maximum. Pourquoi 7 suffisent-ils toujours si on joue bien ? (On y reviendra avec la dichotomie.)

Partie D — Défi ★★

(20 min)

`syracuse.py` — On part d'un entier `n`. S'il est pair on le divise par 2, sinon on le remplace par $3n + 1$. On répète jusqu'à atteindre 1. Afficher toutes les valeurs et le nombre d'étapes (le « temps de vol »). Tester avec 27 (111 étapes!).